



Efficient Dynamic Programming Using Quadrangle Inequalities

F. Frances Yao

Xerox Palo Alto Research Center

Palo Alto, California

ABSTRACT.

Dynamic programming is one of several widely used problem-solving techniques in computer science and operation research. In applying this technique, one always seeks to find speed-up by taking advantage of special properties of the problem at hand. However, in the current state of art, ad hoc approaches for speeding up seem to be characteristic; few general criteria are known. In this paper we give a *quadrangle inequality* condition for rendering speed-up. This condition is easily checked, and can be applied to several apparently different problems. For example, it follows immediately from our general condition that the construction of optimal binary search trees may be speeded up from $O(n^3)$ steps to $O(n^2)$, a result that was first obtained by Knuth using a different and rather complicated argument.

1. INTRODUCTION.

In the application of a general technique, it is often possible to improve the solution by taking advantage of special properties of the problems at hand. *Dynamic programming* is one of several widely used problem-solving techniques in computer science and operation research (see, e.g. [2]). It finds applications in context-free language parsing [8], constructing optimal binary trees [7], finding shortest paths [4], and in solving various "intractible" combinatorial problems (see the references in [2]). In the construction of optimal binary search trees, for example, Knuth[5][7] showed that one can have an algorithm that runs in time $O(n^2)$, whereas straightforward dynamic programming would yield an $O(n^3)$ algorithm. Knuth's proof is quite complicated and involves detailed properties of the optimal binary trees. In general, ad hoc approaches for speeding up seem to be characteristic in dynamic programming; few general criteria are known.

In the present paper we will discuss a *quadrangle inequality* condition for the purpose of achieving speed-up in dynamic programming. This condition is easily checked and will be applied

to several apparently different problems. In particular, it is used to give a simple proof of Knuth's construction of optimal trees, and applied to optimization problems involving t-ary partitions.

2. DYNAMIC PROGRAMMING AND QUADRANGLE INEQUALITIES.

We consider a simple dynamic programming problem for the purpose of illustration.

Example 1. Let $L_1, L_2, \dots, L_n$ be n finite, nonempty sets of strings. We wish to compute their *product (concatenation)* $L_1 \cdot L_2 \cdot \dots \cdot L_n$ by using $L \cdot L'$, the product of two sets, as the primitive. To simplify matters, we assume that the product operation is charged a cost of $|L| \cdot |L'|$, and results in $|L| \cdot |L'|$ strings stored in $L \cdot L'$ (i.e., duplicate strings will not be detected).

Let $|L_i| = n_i$ and $w(i, j) = n_i n_{i+1} \dots n_j$, then the optimal cost $c(i, j)$ for computing $L_i \cdot L_{i+1} \cdot \dots \cdot L_j$ satisfies the following recurrence relations:

$$\begin{aligned} c(i, i) &= 0; \\ c(i, j) &= w(i, j) + \min_{i \leq k \leq j} (c(i, k-1) + c(k, j)) \quad \text{for } i < j. \end{aligned} \tag{1}$$

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1980 ACM 0-89791-017-6/80/0400/0429 \$00.75

We will refer to the function w in the above relations as the *increment function* for c ; it determines the cost function c completely. To evaluate c using the obvious procedure suggested by these equations will require total time $O(n^3)$. However, as we will see, the increment function w in *Example 1* satisfies the *quadrangle inequalities (QI)*

$$w(i, j) + w(i', j') \leq w(i', j) + w(i, j') \quad \text{for } i \leq i' \leq j \leq j'. \quad (2)$$

This property allows the dynamic programming to be speeded up because of the following general theorem.

Theorem 1. If w satisfies QI and furthermore is monotone on the lattice of intervals (ordered by inclusion), i.e.,

$$w(i, j) \leq w(i', j') \quad \text{if } [i, j] \subseteq [i', j'],$$

then the function c defined by (1) can be computed in time $O(n^2)$.

We now verify these conditions for the w in *Example 1*. The monotonicity is obvious. For the QI, let $a = n_i \cdots n_{i'-1}$, $b = n_{i'} \cdots n_j$, and $c = n_{j+1} \cdots n_{j'}$. Then the QI becomes

$$ab + bc \leq b + abc.$$

This is true since

$$0 \leq b(a-1)(c-1).$$

Theorem 1 is proved by establishing the following two lemmas.

Lemma 2.1. If w satisfies QI and is monotone on the lattice of intervals, then the function c defined by (1) also satisfies QI.

Proof. The proof is by induction on the length $l = |j' - i|$ of the "long side" of the quadrangle inequality

$$c(i, j) + c(i', j') \leq c(i', j) + c(i, j') \quad \text{for } i \leq i' \leq j \leq j'. \quad (3)$$

First note that (3) is trivial when $i = i'$ or $j = j'$. Therefore (3) is true when $l \leq 1$. Inductively, consider two cases: A) $i < i' = j < j'$, and B) $i < i' < j < j'$. (See Figure 1).

Case A). $i < i' = j < j'$.

In this case, (3) becomes the (inverse) triangle inequality:

$$c(i, j) + c(j, j') \leq c(i, j') \quad \text{for } i < j < j'. \quad (4)$$

Suppose $c(i, j')$ is minimized at $k = z$; that is, $c(i, j') = c_z(i, j')$ where we use $c_k(i, j)$ to denote $w(i, j) + c(i, k-1) + c(k, j)$. There

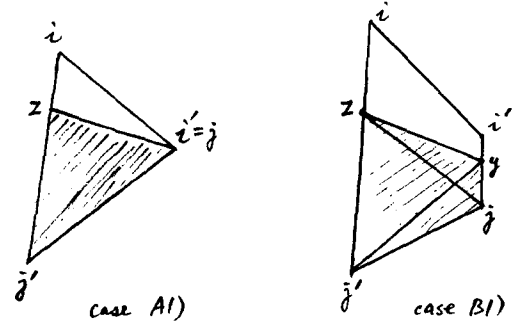


Figure 1. The proof of Lemma 2.1

are two symmetric subcases.

Case A1). $z \leq j$.

We have $c(i, j) \leq c_z(i, j) = w(i, j) + c(i, z-1) + c(z, j)$.

Therefore,

$$\begin{aligned} c(i, j) + c(j, j') &\leq w(i, j) + c(i, z-1) + c(z, j) + c(j, j') \\ &\leq w(i, j') + c(i, z-1) + c(z, j') \\ &= c(i, j'), \end{aligned}$$

where we used the monotonicity of w , and the induction hypothesis (4) at $z \leq j \leq j'$.

Case A2). $z \geq j$. This is symmetric with A1), with all the intervals reversed.

Case B). $i < i' < j < j'$.

Assume the two terms on the right hand side of (3) achieve their values at $k = y$ and $k = z$ respectively. That is,

$$c(i', j) = c_y(i', j), \quad \text{and} \quad c(i, j') = c_z(i, j').$$

We again look at two symmetric subcases.

Case B1). $z \leq y$.

We have

$$c(i', j') \leq c_y(i', j')$$

and

$$c(i, j) \leq c_z(i, j).$$

Adding them up, we obtain

$$\begin{aligned}
& c(i, j) + c(i', j') \\
& \leq c_z(i, j) + c_y(i', j') \\
& = w(i, j) + w(i', j') + c(i, z-1) + c(z, j) + c(i', y-1) + c(y, j') \\
& \quad (5)
\end{aligned}$$

Applying the *QI* of w , and the induction hypothesis (3) at the points $z \leq y < j < j'$, (5) becomes

$$\begin{aligned}
& c(i, j) + c(i', j') \\
& \leq w(i', j) + w(i, j') + c(i, z-1) + c(i', y-1) + c(y, j) + c(z, j') \\
& \leq c_y(i', j) + c_z(i, j') \\
& = c(i', j) + c(i, j')
\end{aligned}$$

Case B2). $z \geq y$. This again reduced to *BI*) when all intervals are reversed. ■

Let us use $K_c(i, j)$ to denote $\max\{k | c_k(i, j) = c(i, j)\}$; so $K_c(i, j)$ is the largest index k where the minimum is achieved in (1). (We define $K_c(i, i) = i$.)

Lemma 2.2. If the function c defined in (1) satisfies *QI*, then we have

$$K_c(i, j) \leq K_c(i, j+1) \leq K_c(i+1, j+1) \quad \text{for } i \leq j. \quad (6)$$

Proof. It is trivially true when $i = j$, therefore assume $i < j$. To prove the first inequality $K_c(i, j) \leq K_c(i, j+1)$, we show that for $i < k \leq k' \leq j$,

$$[c_k(i, j) \leq c_{k'}(i, j)] \Rightarrow [c_k(i, j+1) \leq c_{k'}(i, j+1)]. \quad (7)$$

Take the quadrangle inequality of c at $k \leq k' \leq j < j+1$

$$c(k, j) + c(k', j+1) \leq c(k', j) + c(k, j+1)$$

Adding $w(i, j) + w(i, j+1) + c(i, k-1) + c(i, k'-1)$ to both sides, we get

$$c_k(i, j) + c_{k'}(i, j+1) \leq c_{k'}(i, j) + c_k(i, j+1),$$

from which (7) follows. Similarly, the second inequality $K_c(i, j+1) \leq K_c(i+1, j+1)$ follows from the *QI* of c at $i < i+1 \leq k \leq k'$. ■

Lemma 2.2 says that the matrix $K_c(i, j)$ is nondecreasing along each row and column. As a consequence, when we compute $c(i, j)$ for $\delta = j - i = 0, 1, 2, \dots, n-1$, only $K_c(i+1, j+1) - K_c(i, j)$ minimization operations need to be carried out for

$c(i, j+1)$. Hence for a fixed δ , the total amount of work is $O(n)$; the overall computation time is therefore reduced to $O(n^2)$. This proves *Theorem 1*. ■

We remark that the monotonicity assumption on w in *Lemma 2.1* is necessary for the *QI* of c . For example, if we let $(i, i', j, j') = (1, 2, 2, 3)$, then the *QI* of c becomes

$$c(1, 2) + c(2, 3) \leq c(1, 3),$$

which is equivalent to

$$w(1, 2) + w(2, 3) \leq w(1, 3) + \min(w(1, 2), w(2, 3)),$$

or

$$\max(w(1, 2), w(2, 3)) \leq w(1, 3).$$

3. OPTIMAL BINARY SEARCH TREES.

The construction of optimal binary search trees is a well known example of dynamic programming. The statement of the problem is as follows[5][7].

Example 2. We are given $2n+1$ probabilities $p_1, p_2, \dots, p_n$ and $q_0, q_1, \dots, q_n$ where

p_i = probability that Key_i is the search argument;
 q_i = probability that the search argument
 lies between Key_i and Key_{i+1} .

We wish to find a binary tree which minimizes the expected number of comparisons in the search, namely

$$\begin{aligned}
& \sum_{1 \leq j \leq n} p_j (1 + \text{level of } j\text{th internal node in symmetric order}) + \\
& \sum_{0 \leq k \leq n} q_k (\text{level of the } (k+1)\text{st external node,})
\end{aligned}$$

where the root has level zero.

Let $c(i, j)$ be the cost of an optimal subtree with weights $(p_{i+1}, \dots, p_j; q_i, \dots, q_j)$. Since all subtrees of an optimal tree are optimal, it follows that $c(i, j)$ satisfies the same recurrences as given by Equ.(1) with w now defined by

$$w(i, j) = p_{i+1} + \dots + p_j + q_i + \dots + q_j. \quad (8)$$

This increment function is monotone, and it satisfies the quadrangle inequalities in fact as equalities. It therefore follows from

Theorem 1 that we can have an $O(n^2)$ time construction of an optimal tree by dynamic programming. In [5], the monotone property (6) is derived by a more complex argument.

Note that the question asked in Knuth [7, Section 6.2.2 ex.30] is whether the cost function c satisfies a special case of the quadrangle inequalities, namely

$$c(i, j) + c(i+1, j+1) \leq c(i+1, j) + c(i, j+1), \quad (9)$$

and is therefore answered in the affirmative by *Lemma 2.1*. In fact, (9) is equivalent to the general *QI* since (3) can be derived from (9) by induction on $|i' - i|$ and $|j' - j|$.

4. PATHS IN A CONVEX POLYGON .

We look at an example where the quadrangle inequalities have a most intuitive interpretation, and where binary partitions generalize easily to t -ary partitions.

Example 3. Suppose $v_1 v_2 \dots v_n$ is a convex polygon in E^2 . Let $d(i, j)$ = the Euclidean distance between v_i and v_j if $i \leq j$, and $d(i, j) = 0$ if $i > j$. We notice that d satisfies the *inverse quadrangle inequalities*, i.e.,

$$d(i, j) + d(i', j') \geq d(i', j) + d(i, j') \quad \text{for } i \leq i' \leq j \leq j'. \quad (10)$$

(Inverse *QI*'s are what we need in considering maximization problems such as the present one.) We use $A \otimes B$ to denote the $(\max, +)$ -multiplication of upper triangular matrices A and B . That is, if $A = (a(i, j))$ and $B = (b(i, j))$, then $A \otimes B = (c(i, j))$ where $c(i, j) = \max_{i \leq k \leq j} (a(i, k) + b(k, j))$. We define $D^{(1)} = D = (d(i, j))$, $D^{(t)} = D^{(t-1)} \otimes D$, and write $D^{(t)}$ as $(d^{(t)}(i, j))$. For example, $d^{(2)}(i, j)$ is the length of the longest trajectory from v_i to v_j that allows one bounce off the wall $v_i v_{i+1} \dots v_j$. We are interested in computing $D^{(t)}$ fast.

By associativity $D^{(t)} = D^{(r)} \otimes D^{(s)}$ for $t = r + s$. This multiplication is a special case of a relation of the following form.

$$c(i, j) = w(i, j) + \max_{i \leq k \leq j} (a(i, k) + b(k, j)) \quad \text{for } i \leq j. \quad (11)$$

It follows from *Lemma 4.1* below that $d^{(r)}(i, j)$ satisfies the inverse *QI* for any $r \geq 1$ by induction on r . *Lemma 4.2* then tells us that the multiplication $D^{(r)} \otimes D^{(s)}$ can be done in $O(n^2)$ time for any $r \geq 1$ and $s \geq 1$.

Lemma 4.1 If w , a and b all satisfy the inverse *QI*, then the function c defined by (11) also satisfies the inverse *QI*.

Proof. Similar to the proof of *Lemma 2.1*, except that we need not consider *Case A*) separately from *Case B*).

Lemma 4.2. If both a and b satisfy the inverse *QI*, then for the function c defined by (11) we have

$$K_c(i, j) \leq K_c(i, j+1) \leq K_c(i+1, j+1) \quad \text{for } i \leq j.$$

Proof. Similar to the proof of *Lemma 2.2*.

Theorem 2. For any t , $D^{(t)}$ can be computed in time $O((\log t)n^2)$

Proof. Apply a standard binary algorithm for computing powers. ■

Example 3 is reminiscent of the problem studied in [3], where monotonicity properties similar to *Lemma 4.2* are utilized to find a maximum triangle inscribed in a convex polygon efficiently. An interesting question is whether the present approach can be used to obtain a fast solution for finding maximum k -gons.

5. OPTIMAL T-ARY TREES .

In view of *Theorem 2*, what can we say when *Equ(1)* is generalized to allow $c(i, j)$ to be partitioned into up to t subproblems? The recurrence becomes

$$\begin{aligned} c(i, j) &= w(i, j) + \min_{i < k_1 \leq k_2 \leq \dots \leq k_{t-1} \leq j} (c(i, k_1 - 1) + c(k_1, k_2 - 1) + \dots + c(k_{t-1}, j)) \\ &\quad \text{if } i < j, \\ c(i, j) &= 0 \quad \text{if } i \geq j. \end{aligned} \quad (12)$$

So when $i < j$, the problem of computing $c(i, j)$ is divided into 2 to t subproblems whose sizes are strictly smaller than that of the original problem. (We say a subproblem $c(k, l)$ is *empty* if $k > l$.) This problem is similar to *Example 3*; it requires a little more care since it involves recurrences.

The main result we have here is the following. We say that a function w satisfies the *triangle inequalities (TI)* if

$$w(i, j) + w(j, j') \leq w(i, j') \quad \text{for } i < j < j'.$$

Notice that w satisfies TI implies that w is monotone. (We assume that $w(i, j) \geq 0$.)

Theorem 3. If w satisfies QI and TI , then the function c defined by (12) can be computed in time $O((\log t)n^2)$.

Example 4. Consider the construction of optimal search trees as in *Example 2*, but allowing each node to have degree at most t . In the special case that all the q 's are zero, we have $w(i, j) = p_{i+1} + \dots + p_j$, which satisfies the condition of *Theorem 3*, and such optimal trees can be constructed in time $O((\log t)n^2)$.

Let us denote the 'min' term in Equ(12) by $f^{(t)}(i, j)$. Thus,

$$f^{(t)}(i, j) = \begin{cases} \min_{i < k_1 \leq k_2 \leq \dots \leq k_{t-1} \leq j} (c(i, k_1 - 1) + c(k_1, k_2 - 1) + \dots + c(k_{t-1}, j)) & \text{if } i < j, \\ 0 & \text{if } i \geq j. \end{cases} \quad (13)$$

Furthermore, define $f^{(1)}(i, j) = c(i, j)$; and for $2 \leq q \leq t - 1$, define $f^{(q)}(i, j)$ to be the optimal sum of $\leq q$ subproblems.

$$f^{(q)}(i, j) = \begin{cases} \min_{i \leq k_1 \leq k_2 \leq \dots \leq k_{q-1} \leq j} (c(i, k_1 - 1) + c(k_1, k_2 - 1) + \dots + c(k_{q-1}, j)) & \text{if } i \leq j, \\ 0 & \text{if } i > j. \end{cases} \quad (14)$$

Note that, in a partition of $f^{(q)}(i, j)$, only one subproblem is required to be nonempty.

Fact A. $f^{(1)}(i, j) \geq f^{(2)}(i, j) \geq \dots \geq f^{(t)}(i, j)$.

Proof. All except the last inequality follows immediately from the definition of $f^{(q)}$. If $f^{(t-1)}(i, j)$ is obtained by a decomposition into two or more subproblems, we have $f^{(t-1)}(i, j) \geq f^{(t)}(i, j)$; otherwise $f^{(t-1)}(i, j) = c(i, j) \geq f^{(t)}(i, j)$ by Equ(12) since $w(i, j) \geq 0$. ■

Fact B.

$$f^{(q)}(i, j) = \min_{i \leq k \leq j} (f^{(r)}(i, k - 1) + f^{(s)}(k, j)) \quad \text{for } q = r + s, \\ \text{and } r \geq 1, s \geq 1, 2 \leq q \leq t - 1. \quad (15)$$

Proof. We will show that lefthand side $\geq$ righthand side since the other direction is obvious. If in (14) the minimum value of $f^{(q)}(i, j)$ is achieved with division points $k_1, \dots, k_r, \dots, k_{q-1}$, we choose k to be this k_r on the righthand side of Equ(15). ■

Fact C.

$$f^{(t)}(i, j) = \min_{i < k \leq j} (f^{(r)}(i, k - 1) + f^{(s)}(k, j)) \quad \text{for } t = r + s, \\ \text{and } r \geq 1, s \geq 1. \quad (16)$$

Proof. Similar to the proof of *Fact B*. Again choose k on the righthand side to be the k_r of the lefthand side. ■

Lemma 5.1. In (14), if $f^{(r)}(i, j)$ and $f^{(s)}(i, j)$ satisfy QI for $i - j \leq \delta$, then $f^{(q)}(i, j)$ satisfies QI for $i - j \leq \delta$.

proof. Similar to the proof of *Lemma 2.1*. In the case corresponding to *Case A1*), we need the TI

$$f^{(s)}(z, j) + f^{(q)}(j, j') \leq f^{(s)}(z, j'),$$

which follows from the QI of $f^{(s)}$, and the fact $f^{(q)} \leq f^{(s)}$. Similarly, *Case A2*) follows from the QI of $f^{(r)}$, *Case B1*) from the QI of $f^{(s)}$, and *B2*) from the QI of $f^{(r)}$. (See Figure 2). ■

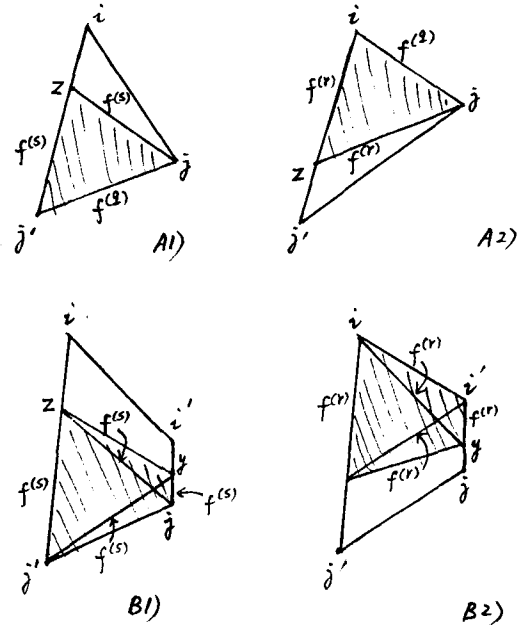


Figure 2. The proof of Lemma 5.1

Lemma 5.2. In (15), if $f^{(r)}(i, j)$ and $f^{(s)}(i, j)$ satisfy QI for $i - j \leq \delta$, then $f^{(t)}(i, j)$ satisfies QI for $i - j \leq \delta + 1$.

proof. Analogous to the proof of *Lemma 5.1*; here the problem sizes are strictly reduced in the inductive step. ■

Lemma 5.3. In (12), w satisfies QI and TI implies that $f^{(1)}(=c), \dots, f^{(q)}, \dots, f^{(t)}$ all satisfy QI .

proof. It follows from the preceding two Lemmas by induction on $\delta = j - i$ and on q , noting that w satisfies QI , TI and $f^{(t)}$ satisfies QI together imply that c satisfies QI . ■

proof of Theorem 3. For the f 's on the lefthand side of (15) and (16), we use $K_f(i, j)$ as before to denote the largest k on the righthand side which allows the minimum value of $f(i, j)$ to be achieved. By the same argument which led to *Lemma 2.2* and 4.2, we have

$$K_f(i, j) \leq K_f(i, j+1) \leq K_f(i+1, j+1) \quad \text{for } i \leq j, \quad (17)$$

and $f = f^{(q)}, 1 \leq q \leq t$.

Let $q_0, q_1, \dots, q_h$ be an *addition chain*[6] for t ; that is, $q_0 = 1$, $q_h = t$, and for each $i \geq 1$, $q_i = q_j + q_k$ for some $j < i, k < i$. It is well known that any t has an addition chain of length $h \leq 2 \log t$. The following procedure then employs Equ(15), (16) and (17) to compute $c(i, j)$.

begin

for $1 \leq i \leq n, 1 \leq m \leq h$ do $f^{(q_m)}(i, i) \leftarrow 0$;

for $\delta \leftarrow 1$ to n do

for $m \leftarrow 1$ to h do

for $j - i = \delta$ do

compute $f^{(q_m)}(i, j)$

end

Because of Equ(17), the innermost loop takes only $O(n)$ steps. Therefore the algorithm uses total time $O(hn^2)$, which is $O((\log t)n^2)$. ■

6. CONCLUDING REMARKS .

In this paper we have considered a general type of conditions which ensures monotonicity of division points in certain dynamic programming processes. This monotonicity property makes it possible to achieve speed-up by a factor of n or more over the straightforward implementations. We would also like to point out some situations where the present results do not apply, and which deserve further study.

The monotonicity property for the division points does not hold for the matrix multiplication chain problem[1], as shown

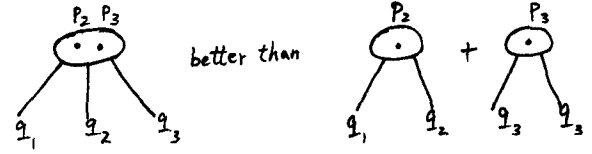


Figure 3a. $c(1,3) + c(2,2) \leq c(1,2) + c(2,3)$.

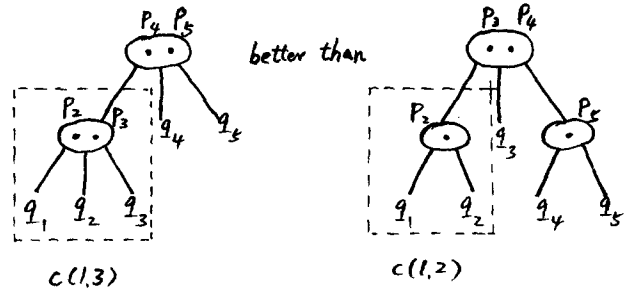
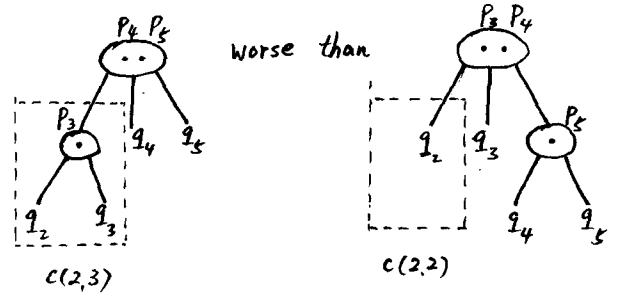


Figure 3b.

For the w defined by (8), Equ (16) fails.

by the following example. Consider the matrices M_1, M_2, M_3, M_4 with dimensions $2 \times 3, 3 \times 2, 2 \times 10$, and 10×1 , respectively. As can be easily verified, the proper order to compute $M_1 M_2 M_3$ is to parenthesize it as $(M_1 M_2) M_3$, while the optimal computation of $M_1 M_2 M_3 M_4$ corresponds to $M_1 (M_2 (M_3 M_4))$.

Similarly, optimal t -ary search trees in general (when the q 's are not zero) do not satisfy the monotonicity property either. The addition of a new leftmost key may force the division points (at the root) to shift rightward! An example is shown in Figure 3. As

the w defined by (8) fails to satisfy TI (for example, $w(1, 2) + w(2, 3) \geq w(1, 3)$), the function c defined by (12) does not satisfy the QI (Figure 3a). When Key_1 is added, the division points change from $\{3, 4\}$ to $\{4, 5\}$. (Figure 3b).

ACKNOWLEDGEMENT

We recently learned that Zhu Yongjin and Wang Jianfang [9], in studying algorithms for constructing alphabetic trees with restricted depth, used an approach similar to ours.

REFERENCES

- [1] A. Aho, J. Hopcroft and J. Ullman, *The design and analysis of computer algorithms*, Addison-Wesley, Reading Mass., 1974.
- [2] K. Q. Brown, *Dynamic programming in computer science*, Computer Science Department Report CMU-CS-79-106, Carnegie-Mellon University, February 1979.
- [3] D. P. Dobkin and L. Snyder, *On a general method for maximizing and minimizing among certain geometric problems*, Proc. IEEE 20th Annual Symposium on Foundations of Computer Science, Puerto Rico, 1979, 9-17.
- [4] R. W. Floyd, *Algorithm 97: shortest path*, *Comm. ACM* 5 (1962), 345.
- [5] D. E. Knuth, *Optimum binary search trees*, *Acta Informatica* 1 (1971), 14-25.
- [6] D. E. Knuth, *The Art of Computer Programming, Vol 2: Seminumerical Algorithms*, Addison-Wesley, Reading Mass., 1975.
- [7] D. E. Knuth, *The Art of Computer Programming, Vol 3: Searching and Sorting*, Addison-Wesley, Reading Mass., 1973.
- [8] D. H. Younger, *Recognition of context-free languages in time n^3* , *Information and Control* 10 (1967), 189-208.
- [9] Y. Zhu and J. Wang, *On alphabetic-extended binary trees with restricted path length*, *Scientia Sinica* 22 (1979), 1362-1371.